Absolutely. If you're teaching **Quick Sort in C**, I'd explain it to students using a simple **"pivot → partition → recursively sort"** approach.

1. The basic idea

Quick Sort works in 3 steps:

1. **Choose a pivot** element.
2. **Partition** the array:
    - Elements smaller than pivot → left
    - Elements greater than pivot → right
3. Recursively apply Quick Sort to the left and right parts.

For example:

```
[7, 2, 1, 6, 8, 5, 3, 4]
```

Choose `4` as pivot:

```
[2, 1, 3] 4 [7, 6, 8, 5]
```

Then sort the two sides again.

2. Simple C program to teach

C/C++

```c
#include <stdio.h>

// Swap two elements
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```c
// Partition the array
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];   // Last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    // Put pivot in its correct position
    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

// Quick Sort function
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        // Find pivot's correct position
        int pi = partition(arr, low, high);

        // Sort left part
```

```
        quickSort(arr, low, pi - 1);

        // Sort right part
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int arr[] = {7, 2, 1, 6, 8, 5, 3, 4};
    int n = sizeof(arr) / sizeof(arr[0]);

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

5. Complexity

Case	Time
Best	`O(n log n)`
Average	`O(n log n)`
Worst	`O(n²)`
Extra space	`O(log n)` average due to recursion

2. Quick Sort in Descending Order

The only major change from ascending Quick Sort is:

```
None
if (arr[j] > pivot)
```

instead of:

```
None
if (arr[j] < pivot)
```

```
C/C++
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
```

```
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        // For descending order
        if (arr[j] > pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
```

```c
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Array in descending order: ");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Input:

```
None
5
10 4 7 2 9
```

Output:

```
None
Array in descending order: 10 9 7 4 2
```

---

3. Quick Sort Using First Element as Pivot

Here we select:

```
C/C++
int pivot = arr[low];
```

The partition logic is slightly different because the pivot is at the **beginning**.

```
C/C++
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
```

```
    int pivot = arr[low];

    int i = low;
    int j = high;

    while (i < j)
    {
        // Find element greater than pivot
        while (arr[i] <= pivot && i < high)
        {
            i++;
        }

        // Find element smaller than pivot
        while (arr[j] > pivot)
        {
            j--;
        }

        if (i < j)
        {
            swap(&arr[i], &arr[j]);
        }
    }

    // Put pivot in its correct position
    swap(&arr[low], &arr[j]);

    return j;
}
```

```c
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");

    for (int i = 0; i < n; i++)
```

```
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Input:

```
None
6
5 2 8 1 9 3
```

Output:

```
None
Sorted array: 1 2 3 5 8 9
```

How to explain the first-element pivot

For:

```
None
5  2  8  1  9  3
↑
pivot = 5
```

We use two pointers:

```
None
i → from left
j → from right
```

Find:

- An element **greater than pivot** from the left.
- An element **smaller than pivot** from the right.
- Swap them.
- Finally, put the pivot in its correct position.

---

4. Quick Sort Without Separate `swap()` Function

Here we don't use:

```C/C++
void swap()
```

Instead, we directly write the swapping code wherever required.

```C/C++
#include <stdio.h>

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            // Swap directly
            int temp = arr[i];
```

```
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    // Put pivot in correct position
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
    int arr[n];

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Input:

None

```
5
10 4 7 2 9
```

Output:

None

```
Sorted array: 2 4 7 9 10
```

Key difference for students

With `swap()` function:

```C/C++
swap(&arr[i], &arr[j]);
```

Without `swap()` function:

```C/C++
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
```

For beginners, I would teach **#4 first**, because they can understand swapping without needing pointers. Then introduce the `swap()` function and explain why `&arr[i]` and `*a` are used.